

Python 程序设计

第 5 讲 (d): Python 模块 module 与包 package

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

什么是 Python 模块?

定义

模块 (**module**) 就是一个 **Python 文件 (.py)**

- 一个模块 = 一个命名空间
- 模块内部可以定义:
 - 函数
 - 变量
 - 类

Python 模块 (Modules)

■ 作用:

- 代码复用
- 命名空间隔离
- 工程化开发基础

■ 模块就像“可重用函数库”，便于组织大型程序

创建模块示例

```
# 文件 mymath.py  
def add(a, b):  
    return a + b  
  
def square(x):  
    return x * x
```

- 一个模块可以包含函数、类和变量
- 保存为 `'mymath.py'` 文件

最基本的 import 方式

- import module
- 使用时必须带模块名前缀

```
import math  
print(math.sqrt(16))
```

import 多个模块

- 想使用 Python 源文件, 只需在另一个源文件里执行 import 语句
- 当解释器遇到 import 语句, 如果模块在当前的搜索路径 (sys.path) 就会被导入

```
import module1[, module2[, ... moduleN]
```

from...import...

```
from Fibo import fib1, fib2
fib1(1000)

result = fib2(1000)
print(result)
```

输出结果为:

1 2 3 5 8 13 21 34 55 89 144 233 377 610 987

[1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987]

from ... import ... as ... 语句: 重命名模块

使用这种方式可以为导入的对象确定一个别名, 可以减少查询次数, 提高访问速度; 同时也可以减少程序员需要输入的代码量

```
from numpy import random as rnd
from math import sin, cos
from random import randint
a = rnd.randint(0,10,(1,3))
b = randint(0, 10)
print(f"sin(a) = {sin(a):.4f}")
print(f"cos(b) = {cos(b):.4f}")
```

创建和导入自定义模块

```
1  # 在mymodule.py中定义函数
2  def greet(name):
3      print(f"Hello, {name}!")
```

```
1  # 在主程序main.py中导入并使用
2  import mymodule
3  mymodule.greet("Alice")
4  # 输出 Hello, Alice!
```

导入模块: 示例

```
import mymath
print(mymath.add(3, 5))
print(mymath.square(4))

from mymath import add
print(add(10, 20))
```

- `'import module'`: 导入整个模块, 用 `'module.func()'` 调用
- `'from module import func'`: 只导入指定函数, 可直接用 `'func()'` 调用

模块搜索路径

■ 当 Python 'import' 模块时, 查找顺序:

- 1 当前目录
- 2 系统环境变量 PYTHONPATH
- 3 Python 内置库路径

■ 查看当前搜索路径:

```
import sys
print(sys.path)
```

模块导入方式: 小结

- 导入多个函数: `from math import sin, cos`
- 导入所有函数 (不推荐): `from math import *`
- 给模块取别名: `import math as m`
- 使用别名提高代码可读性
- 避免命名冲突

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

系统标准模块示例

■ Python 不需要“造轮子”

■ 常用标准模块:

- `math`: 数学函数
- `random`: 随机数
- `datetime`: 日期与时间
- `os, sys`: 系统与环境操作

```
import random  
print(random.randint(1, 10))
```

Python 标准模块 (Standard Library)

- Python 自带的一整套模块集合
- 安装 Python 即可使用
- 特点:
 - 稳定
 - 高质量
 - 跨平台

数学计算: math 模块

■ 提供基础数学函数

```
import math

math.sqrt(16)
math.log(10)
math.sin(math.pi / 2)
```

■ 面向: 确定性数学计算

随机与不确定性: random 模块

```
import random

random.randint(1, 10)
random.choice(["A", "B", "C"])
random.random()
```

- 模拟
- 抽样
- 随机实验

时间处理: time 与 datetime

```
import time  
time.time()  
  
from datetime import datetime  
datetime.now()
```

- 日志记录
- 时间戳
- 程序性能分析

操作系统接口: os 模块

```
import os

os.getcwd()
os.listdir(".")
os.path.exists("data.txt")
```

- 跨平台文件操作
- 路径管理

现代路径接口: pathlib

```
from pathlib import Path

p = Path(".")
p.iterdir()
p / "data.txt"
```

- 面向对象
- 可读性强

系统级交互: sys 模块

```
import sys  
  
sys.version  
sys.path  
sys.exit()
```

- 理解 Python 运行环境
- 与解释器交互

标准模块与第三方模块的关系

■ 标准模块:

- Python 自带
- 无需安装

■ 第三方模块:

- pip 安装
- 如 numpy / pandas

如何系统学习标准模块

- 官方文档 (权威)
- IDE 自动补全
- `help(module)`

```
help(numpy)
help("numpy")
help(numpy.random)
from numpy.random import randint
help(randint)
```

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

模块属性 `__name__`

- 每个模块都有 `__name__` 属性
- 直接运行模块: `__name__ == "__main__"`
- 被导入模块: `__name__ == "模块名"`

```
# mymodule.py
def hello():
    print("Hello")

if __name__ == "__main__":
    hello()
```

- 区分模块是“脚本运行”还是“被导入使用”
- 配合 `main()` 函数思想实现程序入口控制

`__name__` 属性: `if __name__ == '__main__':`

- 一个模块被另一个程序第一次引入时, 其主程序将运行.
- 如果我们想在模块被引入时, 模块中的某一程序块不执行, 可以用 `__name__` 属性来使该程序块仅在该模块自身运行时执行.

```
if __name__ == '__main__':  
    print("==Fibo.py 自身在运行==")  
    fib1(500)  
else:  
    print("==Fibo 被其他模块调用了==")
```

`__name__` 属性: `if __name__ == '__main__':`

```
# 文件名: Fibo.py
def fib1(n): # 定义到不超过 n 的斐波那契数列
    a, b = 1, 1
    print(f'不超过{n}的Fibonacci数组为')
    while b < n:
        print(b, end=' ')
        a, b = b, a+b
    print()
def fib2(n): # 返回到 n 的斐波那契数列
    result = []
    a, b = 1, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
if __name__ == '__main__':
    print("==Fibo.py自身在运行==")
    fib1(500)
else:
    print("==Fibo被其他模块调用了==")
```

dir() 函数

文件名: Fibo.py

def fib1(n): # 定义到不超过 n 的斐波那契数列

a, b = 1, 1

while b < n:

print(b, end=' ')

 a, b = b, a+b

print()

def fib2(n): # 返回到 n 的斐波那契数列

 result = []

 a, b = 1, 1

while b < n:

 result.append(b)

 a, b = b, a+b

return result

dir() 函数-续

```
>>> import Fibo
# 可以使用模块名称来访问函数:
>>> Fibo.fib1(1000)
1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> Fibo.fib2(1000)
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610,
 987]

# 查看模块函数
>>> dir(Fibo)
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'fib1', 'fib2']
```

模块 = 更大粒度的“函数接口”

- 函数是最小抽象
- 模块是函数的集合抽象
- 函数 → 模块 → 标准库
- 学会“调用能力”，而非重复造轮子

当模块数量变多，如何组织模块？

这就需要 **Package** (包)

- 包 (package) 是模块的集合抽象

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

Python 包 (Package) 概述

- 包 (Package) = 一组相关模块的集合
- 本质:
 - 一个目录
 - 用来组织多个模块 (.py 文件)
- 目的:
 - 解决模块数量增多后的“命名混乱”
 - 构建层次化、可维护的大型工程

为什么需要包?

■ 当项目变大:

■ 模块数量迅速增加

- 10 个模块: 还能记住
- 30 个模块: 开始混乱
- 100+ 模块: 无法维护

■ 同名函数、同名模块不可避免

■ 单靠模块会出现的问题:

- 文件平铺在一个目录
- `import` 语句难以管理

包 = 命名空间 + 结构设计

模块 vs 包 (层级关系)

函数 $\subset$ 模块 $\subset$ 包

- 函数: 行为抽象
- 模块: 功能抽象
- 包: 系统抽象

包的基本结构

```
mypackage/  
    __init__.py  
    math_utils.py  
    io_utils.py  
    geometry/  
        __init__.py  
        vector.py
```

- 目录即包
- 包中可以包含:
 - 模块
 - 子包 (嵌套结构)

`__init__.py` 的核心作用

- `__init__.py` 文件: 标志该目录是一个 Python 包
- 在包被导入时 自动执行
- 决定:
 - 包暴露哪些接口
 - 包的初始化行为

包不是“文件夹”，而是“接口单元”

包的导入方式

```
import mypackage.math_utils
mypackage.math_utils.add(3, 5)

from mypackage.math_utils import add
add(3, 5)

from mypackage.geometry.vector import Vector
v = Vector([1, 2])
```

- 包名参与命名空间管理
- 路径即“逻辑结构”

包的 import 路径是如何形成的?

- Python 将目录映射为命名空间
- 使用“点号”访问层级

```
import mypackage.mymodule
```

包的命名空间层次

■ 模块命名方式:

- `package.module`

- `package.subpackage.module`

■ 每一层都是一个独立命名空间

层次结构 = 认知结构

灾难案例: 没有包的工程

- 所有 .py 文件堆在一个目录
- import 路径混乱
- 文件名成了“全局变量”

这是小脚本思维, 不是工程思维

包的设计原则 (工程视角)

- 一个包负责一类功能
- 子包体现逻辑分层
- 接口集中在 `__init__.py`
- 内部实现可以随时重构

包 = 软件架构的最小单位

模块 vs 包 (对比总结)

模块

一个.py 文件

封装函数和类

适合小规模功能

包

一个目录

封装模块和子包

适合系统级组织

包是模块的“上位抽象”

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门: 一个最小可运行的 Python 工程

`__init__.py` 与包接口设计

- `__init__.py` 是包的入口文件
- 决定:
 - 包被 import 时发生什么
 - 包“对外看起来像什么”
- 是 Python 包设计中最核心、最优雅的机制

好的包设计 = 好的 `__init__.py`

回顾: import 一个包时发生了什么?

- 执行该包目录下的 `__init__.py`
- 建立包的命名空间
- 决定哪些名字可以被访问

import 包 $\neq$ import 目录

最简单的 `__init__.py`

```
# mypackage/__init__.py  
print("mypackage imported")
```

```
import mypackage
```

- `__init__.py` 中的代码会被执行
- 包可以拥有“初始化行为”

`__init__.py` 的第一层职责: 初始化

- 配置环境
- 加载依赖模块
- 设置全局常量

初始化 $\neq$ 实现逻辑

__init__.py 的核心职责: 接口聚合

```
# mypackage/math_utils.py  
  
def add(a, b):  
    return a + b
```

```
# mypackage/__init__.py  
  
from .math_utils import add
```

```
from mypackage import add
```

- 用户不必知道模块内部结构
- 接口统一从包层暴露

为什么要“接口聚合”?

- 隐藏内部实现细节
- 减少 import 路径长度
- 降低使用者的认知负担

使用者 $\neq$ 维护者

`__init__.py` = 包的 API 说明书

- 用户只应该接触:
 - **`from package import ...`**
- 不应该直接 `import` 内部模块

直接 `import` 内部模块 = 破坏封装

`__all__`: 显式声明包接口

```
# mypackage/__init__.py
from .math_utils import add
from .geometry import Vector

__all__ = ["add", "Vector"]
```

```
from mypackage import *
```

- `__all__` 明确控制 “可被导出的名字”
- 是接口设计的声明性机制

优秀包的共同特征

- `import` 路径短而稳定
- 内部结构可以随意重构
- `__init__.py` 极其干净

设计包，就是设计边界

小结

- `__init__.py` 是包的“门面”
- 决定包的接口、命名空间与抽象层次
- 是 Python 工程优雅性的集中体现

会写 `__init__.py`, 才算真正会写包

本讲的主要内容

1 1. Python 模块 (module)

- 1.1 创建和导入 (import) 模块
- 1.2 主要系统标准模块
- 1.3 模块属性 `__name__`

2 2. Python 包 (Package)

- 2.1 什么是包 (Package)
- 2.2 `__init__.py` 与包接口设计
- 2.3 `__name__ == "__main__"` 与相对导入

3 3. 工程化入门：一个最小可运行的 Python 工程

`__name__ == "__main__"` 与相对导入

- Python 中最容易 “用会、但说不清” 的机制
- 同时涉及:
 - 模块运行方式
 - 包的上下文环境
 - `import` 的底层规则

灾难案例: 为什么这段代码有时能跑, 有时报错?

```
from .math_utils import add  
  
print(add(3, 5))
```

- 在某些情况下运行正常
- 在“直接运行”时直接报错

初学者最常见的困惑来源

先解决一个根本问题: 模块是怎么运行的?

■ Python 模块有两种身份:

- 1 被导入的模块
- 2 被直接运行的脚本

身份不同, `__name__` 就不同

`__name__` 的规则

```
# example.py  
print(__name__)
```

■ 直接运行:

- `__name__ == "__main__"`

■ 被 import:

- `__name__ == " 模块名 "`

`__main__` 的真实含义

- `__main__` 不是文件名
- 表示: 当前解释器执行的“入口模块”
- 每次运行程序, 只会有一个 `__main__`

`__main__` = 程序起点

标准用法: 主程序保护

```
def main():  
    print("program running")  
  
if __name__ == "__main__":  
    main()
```

- 直接运行: 执行 main()
- 被 import: 不执行 main()

这一模式解决了什么问题?

- 防止 import 时执行测试代码
- 支持模块“既能用，又能跑”
- 是工程代码的必备结构

模块 $\neq$ 脚本，但可以二者兼得

现在回到: 什么是相对导入?

- 相对导入使用:
 - . 当前包
 - .. 上一级包
- 只能用于包内部

相对导入依赖 “包上下文”

相对导入示例

```
# mypackage/math_utils.py  
from .helpers import check_number
```

- “.” 表示当前包
- Python 需要知道当前模块属于哪个包

关键结论: 为什么不能直接运行含相对导入的模块?

- 直接运行时:
 - 该文件成为 `__main__`
 - 不再属于任何包
- 相对导入失去参照物

`__main__` 没有“父包”

一句话讲清楚本质

相对导入 $\neq$ 文件路径

相对导入 = 包内逻辑关系

正确的工程运行方式

- 从包外运行入口脚本
- 或使用:

```
python -m mypackage.main
```

- 让 Python 知道包结构

工程级黄金规则

- 包内模块: 使用相对导入
- 包外入口: 使用绝对导入
- 永远保留 `__main__` 保护

规则一旦统一, import 再无玄学

小结

- `__name__` 决定模块身份
- `__main__` 是程序唯一入口
- 相对导入依赖包上下文
- “不能直接运行”不是缺陷，而是设计选择

最小可运行 Python 工程: 概述

- 本节目标不是“写代码”
- 而是回答一个问题:

一个“规范的 Python 程序”到底长什么样?

从脚本思维到工程思维

■ 初学者常见模式:

- 一个.py 文件
- 从上写到下

■ 工程化模式:

- 明确入口
- 功能分层
- 可扩展、可维护

脚本能跑 工程可用

最小工程目录结构

```
project/  
  
    main.py  
    mypackage/  
        __init__.py  
        math_utils.py  
        vector.py
```

- 只有两层
- 但五脏俱全

每一部分各司其职

■ main.py

- 程序入口
- 负责“组织流程”

■ mypackage/

- 业务逻辑
- 可被复用、测试、扩展

入口与实现必须分离

实现模块: math_utils.py

```
def add(a, b):  
    return a + b  
  
def square(x):  
    return x * x
```

- 只写“功能”
- 不关心谁来调用

实现模块: vector.py

```
class Vector:
    def __init__(self, data):
        self.data = data

    def norm(self):
        return sum(x*x for x in self.data) **
            0.5
```

■ 类只关心自身状态与行为

设计包接口: `__init__.py`

```
from .math_utils import add, square
from .vector import Vector

__all__ = ["add", "square", "Vector"]
```

- 对外暴露统一接口
- 隐藏内部结构

此时包对用户“看起来像什么”?

用户只需记住:

```
from mypackage import add, Vector
```

使用者不需要知道模块结构

程序入口: main.py

```
from mypackage import add, Vector

def main():
    print(add(3, 5))
    v = Vector([3, 4])
    print(v.norm())

if __name__ == "__main__":
    main()
```

■ 明确入口

■ 明确执行逻辑

为什么 main.py 必须在包外?

- main.py 是“使用者视角”
- 包是“功能提供者视角”
- 避免相对导入问题

入口文件不属于包

工程运行方式

■ 推荐方式:

```
python main.py
```

■ 所有 import 行为清晰可控

这一结构解决了哪些真实问题?

- import 不再混乱
- 功能可复用
- 结构可扩展
- 测试可插入

这是工业界最常见的基础结构

现在应该具备的能力

- 看懂一个 Python 项目结构
- 知道入口在哪里
- 知道逻辑代码该写在哪里

已经跨过“只会写脚本”的门槛

小结

- 工程 = 结构 + 规则
- main.py 定义 “怎么跑”
- 包定义 “能做什么”
- `__init__.py` 定义 “怎么用”

Python 工程思维至此闭环

Thanks!